

Lab 7 report

PB24111630 黄雯佩

实验目的与内容

学习cache基本原理

实现LRU、FIFO、Random替换策略的cache

实现N路组相连的LRU替换策略cache

二路组相连LRU Cache

1. 关键代码

```

// Cache脏位 - Way 0
wire dirty_way0; // Cache脏位 - Way 1
wire dirty_way1; // Cache脏位 - Way 1
wire lru; // LRU位: 0表示Way0是LRU, 1表示Way1是LRU
wire victim_way; // 当前miss时应替换的路: 优先无效路, 否则选择LRU路

```

simple cache基础上增加LRU标志位

```

// LRU相关辅助
wire dirty_lru_way_read; // 当前要替换的LRU路的脏位 (READ)
wire valid_lru_way_read; // 当前要替换的LRU路的有效位 (READ)
wire dirty_lru_way_write; // 当前要替换的LRU路的脏位 (WRITE)
wire valid_lru_way_write; // 当前要替换的LRU路的有效位 (WRITE)

assign victim_way = !valid_way0 ? 1'b0 :
                    !valid_way1 ? 1'b1 :
                    lru;

assign dirty_lru_way_read = victim_way ? dirty_way1 : dirty_way0;
assign valid_lru_way_read = victim_way ? valid_way1 : valid_way0;
assign dirty_lru_way_write = victim_way ? dirty_way1 : dirty_way0;
assign valid_lru_way_write = victim_way ? valid_way1 : valid_way0;

```

未命中时的替换逻辑

2. 仿真

```
// 检测测试完成, 输出结果
always @(posedge clk) begin
    if (test_cnt == READ_NUM + WRITE_NUM) begin
        test_done <= 1;
        // 计算命中次数 = 总访问次数 - 缺失次数
        hit_cnt <= (READ_NUM + WRITE_NUM) - miss_cnt;
        if (diff == 0) begin
            $display("===== %d way TEST PASSED =====",WAY_NUM);
        end else begin
            $display("===== %d way TEST FAILED =====",WAY_NUM);
        end
        $display("===== PERFORMANCE REPORT =====");
        $display("Total Accesses : %4d", READ_NUM + WRITE_NUM);
        $display("Hit Count      : %4d", (READ_NUM + WRITE_NUM) - miss_cnt);
        $display("Miss Count     : %4d", miss_cnt);
        $display("Hit Rate       : %4d / %4d = %d.%02d%%",
            (READ_NUM + WRITE_NUM) - miss_cnt,
            READ_NUM + WRITE_NUM,
            (((READ_NUM + WRITE_NUM) - miss_cnt) * 10000) / (READ_NUM + WRITE_NUM) / 100,
            (((READ_NUM + WRITE_NUM) - miss_cnt) * 10000) / (READ_NUM + WRITE_NUM) % 100);
        $display("Total Cycles   : %d", total_cycles);
        $display("IPC (ideal)    : %d / %d = %d.%02d%%",
            READ_NUM + WRITE_NUM,
            total_cycles,
            ((READ_NUM + WRITE_NUM) * 10000) / total_cycles / 100,
            ((READ_NUM + WRITE_NUM) * 10000) / total_cycles % 100);
        $display("=====");
    end
    $finish;
end
```

仿真结束后输出命中率和周期数, 便于比较

二路组相连多样化替换策略

1. FIFO

先入先出

2. Random

多项式多项式: $x^4 + x + 1$ 的伪随机替换

3. 仿真结果

```
WARNING: [Wavedata 42-489] Can't add object /cache_tb/test_data to the Wave window because it has 990000 bits, which exceeds
# run 1000000ns
===== TEST PASSED =====
===== PERFORMANCE REPORT =====
Total Accesses : 3000
Hit Count      : 2119
Miss Count     : 881
Hit Rate       : 2119 / 3000 =      70.63%
Total Cycles   :    13864
IPC (ideal)    :    3000 /    13864 =    21.63%
=====
$finish called at time : 27730 ns : File "E:/CoursesData/COD lab/lab7_t/src/cache_tb.v" Line 6171
INFO: [USF-XSim-96] XSim completed. Design snapshot 'cache_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000000ns
launch_simulation: Time (s): cpu = 00:00:01 ; elapsed = 00:00:08 . Memory (MB): peak = 1521.039 ; gain = 0.000
```

Type a Tcl command here

LRU

Tcl Console x Messages Log Linter

```
# run 10000ns
===== TEST PASSED =====
===== PERFORMANCE REPORT =====
Total Accesses :      3000
Hit Count      :      2111
Miss Count     :       889
Hit Rate       :      2111 /      3000 =      70.36%
Total Cycles   :     13940
IPC (ideal)    :      3000 /     13940 =     21.52%
=====

$finish called at time : 27882 ns : File "E:/CoursesData/COD lab/lab7_t/src/cache_tb.v" Line 6171
INFO: [USF-XSim-96] XSim completed. Design snapshot 'cache_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 10000ns
launch_simulation: Time (s): cpu = 00:00:02 ; elapsed = 00:00:10 . Memory (MB): peak = 2440.062 ; gain = 0.000
```

FIFO

```
# run 10000ns
===== TEST PASSED =====
===== PERFORMANCE REPORT =====
Total Accesses : 3000
Hit Count      : 2097
Miss Count     : 903
Hit Rate       : 2097 / 3000 =      69.90%
Total Cycles   :   14052
IPC (ideal)    :    3000 /   14052 =     21.34%
=====

$finish called at time : 28106 ns : File "E:/CoursesData/COD lab/lab7_t/src/cache_tb.v" Line 6171
INFO: [USF-XSim-96] XSim completed. Design snapshot 'cache_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 10000ns
launch_simulation: Time (s): cpu = 00:00:02 ; elapsed = 00:00:08 . Memory (MB): peak = 2440.062 ; gain = 0.000
```

random

4. 结果比较

替换策略	访问总数	命中次数 (Hit)	未命中次数 (Miss)	命中率 (Hit Rate)	总周期数 (Total Cycles)	IPC (理想)
LRU	3000	2119	881	70.63%	13864	21.63%
FIFO	3000	2111	889	70.36%	13940	21.52%
Random	3000	2097	903	69.90%	14052	21.34%

LRU准确捕获时间局部性，命中率最高且2路组下硬件开销极小，但大路组数下控制逻辑复杂。

FIFO 实现最简单，但盲目看重进入先后易误杀核心热点。

Random 硬件开销低，高路数下性价比高，但 2路组下随机性强，命中率最低。

N路组相连cache

1. 核心代码

```

generate //tag+data+LRU
for (gi = 0; gi < WAY_NUM; gi = gi + 1) begin : gen_way // 每一路
    bram #(
        .ADDR_WIDTH(INDEX_WIDTH),
        .DATA_WIDTH(TAG_META_WIDTH)
    ) tag_bram (
        .clk(clk),
        .raddr(r_index),
        .waddr(w_index),
        .din({w_valid, w_dirty, tag}),
        .we(tag_we[gi]),
        .dout(tag_meta_dout_bus[gi*TAG_META_WIDTH +: TAG_META_WID //状态转换
    );

    bram #(
        .ADDR_WIDTH(INDEX_WIDTH),
        .DATA_WIDTH(LINE_WIDTH)
    ) data_bram (
        .clk(clk),
        .raddr(r_index),
        .waddr(w_index),
        .din(w_line),
        .we(data_we[gi]),
        .dout(data_dout_bus[gi*LINE_WIDTH +: LINE_WIDTH])
    );

    bram #(
        .ADDR_WIDTH(INDEX_WIDTH),

```

循环generate出N way个BRAM，分别存储tag、LRU和data

```

for (i = 0; i < WAY_NUM; i = i + 1) begin //命中检测
    if (!hit_r && //防止多路命中保护
        tag_meta_dout_bus[i*TAG_META_WIDTH + TAG_WIDTH + 1] && //有效位
        (tag_meta_dout_bus[i*TAG_META_WIDTH +: TAG_WIDTH] == tag)) begin
        hit_r = 1'b1;
        hit_way_r = i[WAY_INDEX_WIDTH-1:0];
    end
end

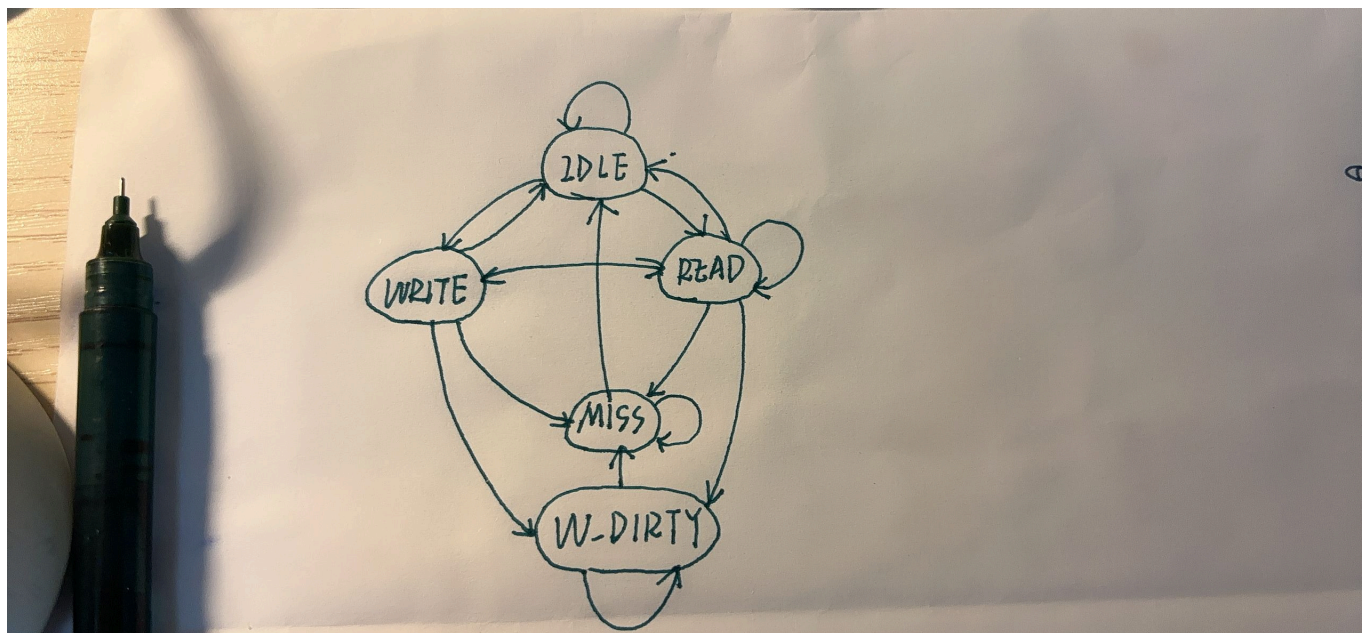
for (i = 0; i < WAY_NUM; i = i + 1) begin //寻找victim
    if (!invalid_found_r &&
        !tag_meta_dout_bus[i*TAG_META_WIDTH + TAG_WIDTH + 1]) begin //找到无效路
        victim_way_r = i[WAY_INDEX_WIDTH-1:0];
        victim_line_r = data_dout_bus[i*LINE_WIDTH +: LINE_WIDTH];
        victim_tag_r = tag_meta_dout_bus[i*TAG_META_WIDTH +: TAG_WIDTH];
        victim_valid_r = 1'b0;
        victim_dirty_r = 1'b0;
        victim_stamp_r = lru_dout_bus[i*LRU_WIDTH +: LRU_WIDTH];
        invalid_found_r = 1'b1;
    end
    else if (!invalid_found_r &&
        lru_dout_bus[i*LRU_WIDTH +: LRU_WIDTH] < victim_stamp_r) begin //时间戳更小
        victim_way_r = i[WAY_INDEX_WIDTH-1:0];
        victim_line_r = data_dout_bus[i*LINE_WIDTH +: LINE_WIDTH];
        victim_tag_r = tag_meta_dout_bus[i*TAG_META_WIDTH +: TAG_WIDTH];
        victim_valid_r = 1'b1;
        victim_dirty_r = tag_meta_dout_bus[i*TAG_META_WIDTH + TAG_WIDTH];
        victim_stamp_r = lru_dout_bus[i*LRU_WIDTH +: LRU_WIDTH];
    end
end

```

命中检测和替换路选择逻辑

使用基本的时间戳标记使用先后，体现多路组情况下的LRU策略空间开销大

2. 简要状态转换图；



3. 仿真结果

```

WARNING: [Wavedata 42-489] Can't add object "/cache_tb/test_data" to the wave window because it has 99000 bits, which
# run 1000000ns
=====
                2 way TEST PASSED =====
===== PERFORMANCE REPORT =====
Total Accesses : 3000
Hit Count      : 2113
Miss Count     : 887
Hit Rate       : 2113 / 3000 =          70.43%
Total Cycles   : 13918
IPC (ideal)    : 3000 / 13918 =          21.55%
=====
$finish called at time : 27838 ns : File "E:/CoursesData/COD lab/lab7_t/src/cache_tb.v" Line 6171
INFO: [USF-XSim-96] XSim completed. Design snapshot 'cache_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000000ns
launch_simulation: Time (s): cpu = 00:00:02 ; elapsed = 00:00:08 . Memory (MB): peak = 1515.648 ; gain = 2.102

```

```

WARNING: [Wavedata 42-489] Can't add object "/cache_tb/test_data" to the wave window because it has 99000 bits, wh
# run 1000000ns
=====
                4 way TEST PASSED =====
===== PERFORMANCE REPORT =====
Total Accesses : 3000
Hit Count      : 2159
Miss Count     : 841
Hit Rate       : 2159 / 3000 =          71.96%
Total Cycles   : 13424
IPC (ideal)    : 3000 / 13424 =          22.34%
=====
$finish called at time : 26850 ns : File "E:/CoursesData/COD lab/lab7_t/src/cache_tb.v" Line 6171
INFO: [USF-XSim-96] XSim completed. Design snapshot 'cache_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000000ns
launch_simulation: Time (s): cpu = 00:00:02 ; elapsed = 00:00:08 . Memory (MB): peak = 1508.227 ; gain = 0.711

```

```

=====
                8 way TEST PASSED =====
===== PERFORMANCE REPORT =====
Total Accesses : 3000
Hit Count      : 2207
Miss Count     : 793
Hit Rate       : 2207 / 3000 =          73.56%
Total Cycles   : 12794
IPC (ideal)    : 3000 / 12794 =          23.44%
=====
$finish called at time : 25590 ns : File "E:/CoursesData/COD lab/lab7_t/src/cache_tb.v" Line 6171
INFO: [USF-XSim-96] XSim completed. Design snapshot 'cache_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000000ns
launch_simulation: Time (s): cpu = 00:00:02 ; elapsed = 00:00:08 . Memory (MB): peak = 1512.246 ; gain = 2.805

```

```
===== 16 way TEST PASSED =====
===== PERFORMANCE REPORT =====
Total Accesses : 3000
Hit Count      : 2289
Miss Count     : 711
Hit Rate       : 2289 / 3000 =      76.30%
Total Cycles   :      11700
IPC (ideal)    :      3000 /      11700 =      25.64%
=====
$finish called at time : 23402 ns : File "E:/CoursesData/COD lab/lab7_t/src/cache_tb.v" Line 6171
INFO: [USF-XSim-96] XSim completed. Design snapshot 'cache_tb_behav' loaded.
INFO: [USF-XSim-97] XSim simulation ran for 1000000ns
launch_simulation: Time (s): cpu = 00:00:02 ; elapsed = 00:00:08 . Memory (MB): peak = 1513.547 ; gain = 1.301
```

4. 结果分析

相联度 N (Way)	访问总数	命中次数 (Hit)	未命中次数 (Miss)	命中率 (Hit Rate)	总周期数 (Total Cycles)	IPC (理想)
2 路	3000	2113	887	70.43%	13918	21.55%
4 路	3000	2159	841	71.96%	13424	22.34%
8 路	3000	2207	793	73.56%	12794	23.44%
16 路	3000	2289	711	76.30%	11700	25.64%

随着组相联度N的翻倍递增，Cache 命中率显著上升，总执行周期大幅下降，IPC 稳步提升